

Universidad CENFOTEC

Escuela de Ingeniería del Software

Especificación de Diseño de Software

CampusFest — Gestión del Festival Estudiantil

Versión 2.0 (final)

Proyecto Integrador 1 (SOFT-11) · Sección SCV0 · Periodo 2026-C2

Grupo 3 — Federico Barón Montoya · Steven Méndez Jiménez · Mariana Soto Castro

Profesora: Verónica Isabel Mora Lezcano

11 de agosto de 2026

Historial de versiones

Versión	Fecha	Autor	Cambios principales
1.0	28/06/2026	Grupo 3	Versión inicial. Arquitectura, modelo de datos preliminar, tecnologías, casos de uso, diseño de interfaz y flujo de navegación (Bitácora 2).
1.1	19/07/2026	Grupo 3	Ajustes del Sprint 3: mecanismo de autenticación sin contraseña, estructura de carpetas del backend y definición inicial de los endpoints.
2.0	11/08/2026	Grupo 3	Versión final. Arquitectura verificada contra la implementación; diseño detallado de las seis colecciones con campo y tipo de dato; catálogo completo de la API REST; estrategia de validación y manejo de errores; estructura definitiva de carpetas; incorporación del módulo de Contacto; matriz de trazabilidad requisito → endpoint → caso de prueba.

1. Introducción

1.1 Propósito

Esta Especificación de Diseño de Software (EDS) documenta cómo está construido el sistema CampusFest. Mientras la ERS establece qué debe hacer el sistema, este documento describe la solución técnica: su arquitectura, el diseño de la base de datos, la interfaz de programación, la estructura del código, la estrategia de validación y manejo de errores, y el flujo de navegación entre pantallas.

La versión 2.0 corresponde al cierre del proyecto y describe el sistema tal como está implementado, no como se planeó inicialmente. Cada decisión de diseño se mantiene trazable respecto a los requisitos de la ERS v2.0.

1.2 Alcance del diseño

El diseño cubre los dos ámbitos del sistema y sus seis módulos funcionales: autenticación, actividades, inscripciones, stands, agenda y contacto, más el registro de resultados. Comprende la capa de presentación (páginas públicas y panel administrativo), la capa de servicios (API REST) y la capa de persistencia (base de datos documental).

1.3 Documentos de referencia

- **ERS v2.0:** especificación de requisitos, versión final.
- **Plan de Pruebas:** plan de pruebas del Sprint 4, con 47 casos que cubren la totalidad de los módulos.
- **Matriz de Trazabilidad v3.0:** relación entre requisitos, endpoints y casos de prueba.
- **Bitácoras 1 a 4:** registros de los cuatro sprints del proyecto.

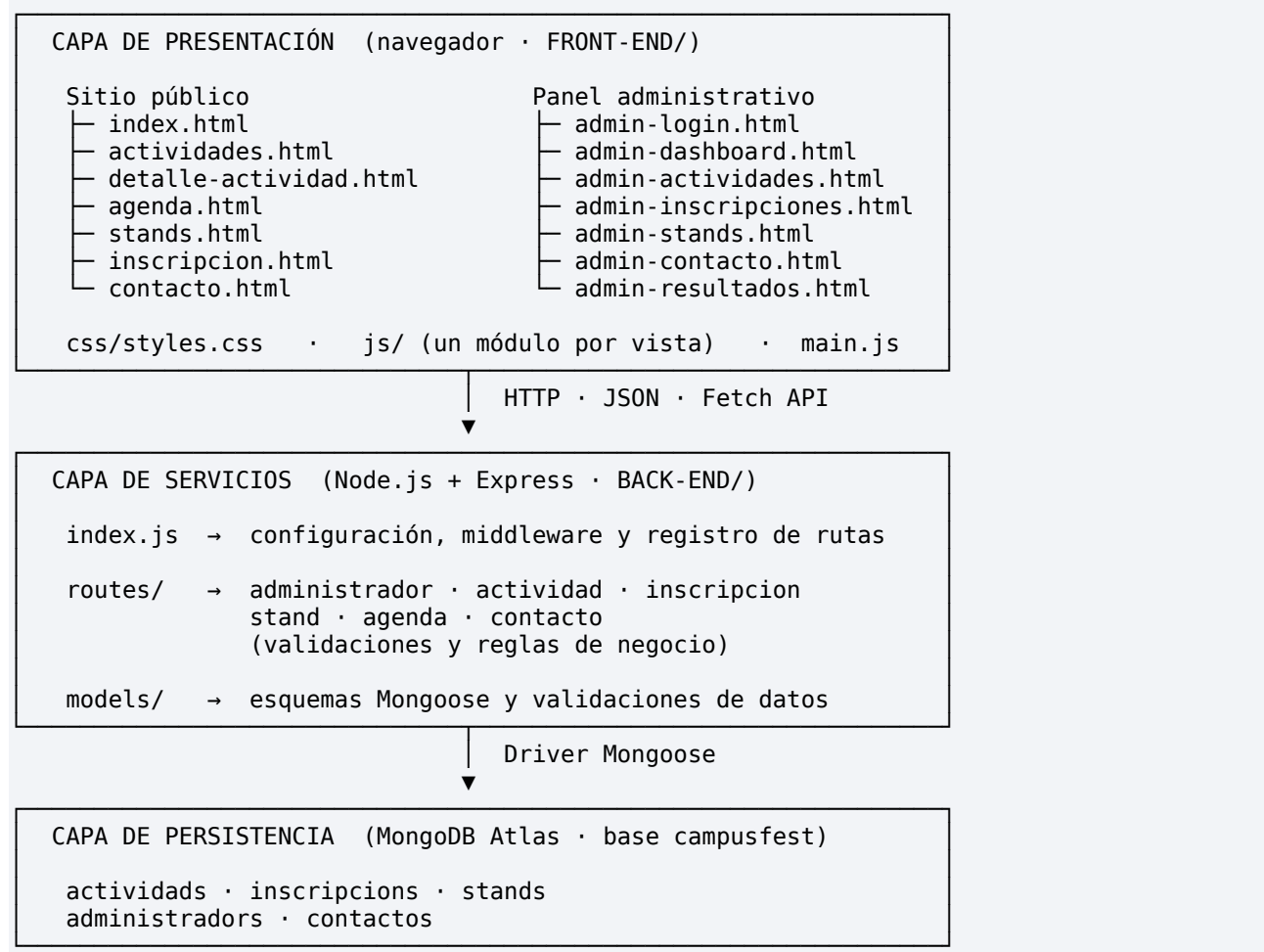
2. Arquitectura del sistema

2.1 Estilo arquitectónico

CampusFest se estructura como una aplicación web cliente-servidor de tres capas, con separación estricta entre presentación, lógica de negocio y persistencia. El cliente y el servidor se comunican exclusivamente mediante peticiones HTTP contra una API REST que intercambia documentos JSON.

Esta separación permite que ambas capas evolucionen de forma independiente: la interfaz puede modificarse sin alterar las reglas de negocio, y el servidor puede atender a otros clientes en el futuro sin cambios estructurales. Responde directamente a RNF-MAN-01 (organización modular por responsabilidades) y a RNF-02 (consumo de la información mediante la API).

Diagrama de capas



2.2 Patrones de diseño aplicados

Patrón	Aplicación en el sistema	Beneficio
Separación en capas	Presentación (HTML/CSS/JS), servicios (Express) y persistencia (MongoDB) residen en ámbitos independientes que se comunican por contratos definidos.	Permite modificar la interfaz sin tocar las reglas de negocio y facilita la localización de defectos.

Patrón	Aplicación en el sistema	Beneficio
Modelo-Ruta (variante de MVC)	Cada entidad tiene un modelo que define su estructura y validaciones, y una ruta que concentra sus operaciones y reglas de negocio. La vista reside en el cliente.	Un mismo modelo se reutiliza desde cualquier ruta; las reglas de negocio quedan en un único lugar por entidad.
API REST	Los recursos se identifican por la ruta, la operación se expresa mediante el método HTTP y el resultado mediante el código de estado.	Contrato predecible y uniforme para el cliente; facilita las pruebas de cada operación de forma aislada.
Repositorio (mediante Mongoose)	El acceso a datos se realiza a través de los modelos de Mongoose, que abstraen las operaciones sobre la base de datos.	El código de negocio no manipula consultas nativas; las validaciones de estructura se centralizan en el esquema.
Middleware en cadena	express.json, body-parser y cors procesan cada petición antes de llegar a la ruta correspondiente.	El tratamiento común de las peticiones no se repite en cada ruta.
Módulo por vista	Cada página del cliente carga un único archivo JavaScript responsable de su comportamiento, más un módulo común de utilidades.	Evita colisiones entre vistas y mantiene acotado el alcance de cada cambio.
Referencia entre documentos	Las inscripciones referencian su actividad y los stands referencian sus actividades vinculadas mediante identificadores, resueltos con populate al consultar.	Evita la duplicación de información y mantiene la coherencia cuando una actividad cambia.

2.3 Flujo de una operación

A modo de ilustración, el registro de una inscripción recorre las tres capas de la siguiente forma:

- **1. Presentación:** El estudiante completa el formulario en `inscripcion.html`. El módulo `js/inscripcion.js` valida que los campos obligatorios estén presentes y que el correo pertenezca al dominio institucional, y envía una petición POST a `/inscripcion`.
- **2. Servicios:** La ruta `inscripcion.route.js` verifica que la actividad exista, que no esté cancelada, que tenga cupos disponibles, que falten más de 36 horas para su inicio, que el correo sea institucional y que el estudiante no esté ya inscrito. Si alguna condición no se cumple, responde con el código 400 y un mensaje explicativo.
- **3. Persistencia:** Superadas las validaciones, se crea el documento de inscripción y se decrementa el contador de cupos disponibles de la actividad, actualizando su estado a “Lleno” si el contador llega a cero.
- **4. Respuesta:** El cliente recibe el código 201 y muestra la notificación de confirmación en pantalla, según exige RF-INS-06.

3. Diseño de la base de datos

3.1 Consideraciones generales

La base de datos campusfest reside en MongoDB Atlas y contiene cinco colecciones. Se optó por un modelo documental con referencias (no incrustación) entre actividades e inscripciones, y entre stands y actividades, porque una actividad es consultada de forma independiente y sus datos cambian con frecuencia: incrustarla duplicaría información y obligaría a actualizaciones en cascada.

Mongoose deriva el nombre de cada colección pluralizando el nombre del modelo. Todos los documentos incluyen automáticamente el identificador `_id` de tipo `ObjectId`, generado por la base de datos.

3.2 Colección actividades

Almacena las actividades del festival. Es la entidad central del sistema: las inscripciones y los stands la referencian.

Campo	Tipo	Obligatorio	Descripción y reglas
<code>_id</code>	<code>ObjectId</code>	Automático	Identificador único generado por la base de datos.
<code>nombreActividad</code>	<code>String</code>	Sí	Nombre con el que la actividad se publica.
<code>categoriaActividad</code>	<code>String</code>	Sí	Categoría temática de la actividad (cultural, deportiva, tecnológica, artística, gastronómica, recreativa).
<code>descripcionActividad</code>	<code>String</code>	Sí	Descripción ampliada que se muestra en el detalle.
<code>requisitosActividad</code>	<code>String</code>	No	Requisitos de participación. Si se omite, el detalle indica que no hay requisitos especiales.
<code>fechaActividad</code>	<code>Date</code>	Sí	Fecha de realización. Base del ordenamiento de la agenda y del cálculo del cierre de inscripciones.
<code>horaActividad</code>	<code>String</code>	Sí	Hora de inicio en formato HH:MM.
<code>lugarActividad</code>	<code>String</code>	Sí	Ubicación física de la actividad.
<code>esVirtual</code>	<code>Boolean</code>	Sí	Indica si la actividad se realiza de forma virtual.
<code>enlaceActividad</code>	<code>String</code>	Condicional	Enlace de acceso. Obligatorio únicamente cuando <code>esVirtual</code> es verdadero (validación condicional en el esquema).
<code>cupoMaximoActividad</code>	<code>Number</code>	Sí	Número máximo de participantes definido por el administrador.
<code>estadoActividad</code>	<code>String</code>	Sí	Estado de la actividad. Valores admitidos: Disponible, Lleno, Cancelado. Valor por omisión: Disponible.
<code>cuposDisponiblesActividad</code>	<code>Number</code>	Automático	Cupos restantes. Se inicializa con el valor de <code>cupoMaximoActividad</code> y se decrementa con cada inscripción.

3.3 Colección inscripciones

Registra la inscripción de un estudiante en una actividad. Referencia la actividad correspondiente.

Campo	Tipo	Obligatorio	Descripción y reglas
<code>_id</code>	<code>ObjectId</code>	Automático	Identificador único generado por la base de datos.
<code>nombreCompleto</code>	<code>String</code>	Sí	Nombre completo del estudiante.

Campo	Tipo	Obligatorio	Descripción y reglas
identificacion	String	Sí	Número de identificación. Junto con la actividad, determina si existe una inscripción duplicada (RF-INS-05).
correo	String	Sí	Correo del estudiante. Debe pertenecer al dominio @ucenfotec.ac.cr (RNF-03).
telefono	String	Sí	Número de teléfono de contacto.
carrera	String	Sí	Carrera que cursa el estudiante.
actividadSeleccionada	ObjectId (ref: actividad)	Sí	Referencia a la actividad en la que se inscribe. Se resuelve con populate para mostrar su nombre.
comentarioOpcional	String	No	Observación adicional del estudiante.
fechaInscripcion	Date	Automático	Marca temporal del registro. Se asigna con la fecha y hora actuales (RF-INS-11).

3.4 Colección stands

Almacena los stands o grupos participantes y sus actividades vinculadas.

Campo	Tipo	Obligatorio	Descripción y reglas
_id	ObjectId	Automático	Identificador único generado por la base de datos.
nombreStand	String	Sí	Nombre del stand, grupo o club.
categoriaStand	String	Sí	Categoría del stand. Valores admitidos: Académico, Deportivo, Cultural, Tecnológico, Emprendimiento.
responsableStand	String	Sí	Persona responsable del stand.
ubicacionStand	String	Sí	Ubicación asignada dentro del campus.
descripcionStand	String	No	Descripción de la propuesta del stand.
actividadesStand	Array de ObjectId (ref: actividad)	No	Actividades relacionadas con el stand (RF-STA-05). Se resuelven con populate para mostrar sus nombres.

3.5 Colección administradores

Registro de los correos autorizados para acceder al panel administrativo.

Campo	Tipo	Obligatorio	Descripción y reglas
_id	ObjectId	Automático	Identificador único generado por la base de datos.
correoAdministrador	String	Sí	Correo institucional del administrador. Único, se almacena en minúscula y sin espacios.
nombreAdministrador	String	No	Nombre del integrante del comité organizador.

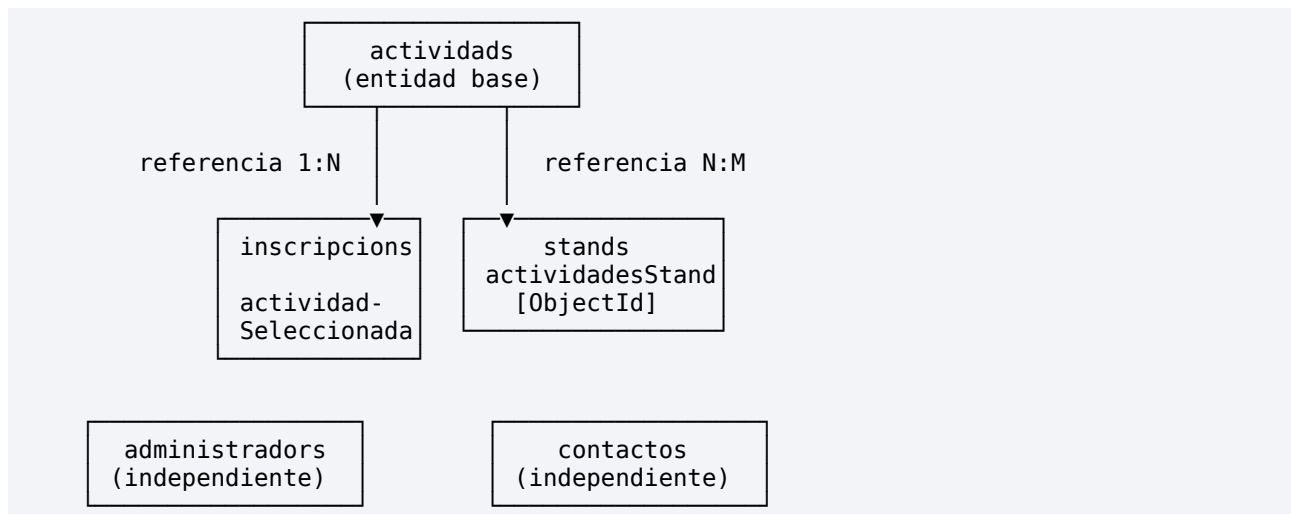
Observación de diseño: esta colección está definida pero el procedimiento de inicio de sesión no la consulta; la validación se limita al dominio del correo. La consecuencia se documenta como riesgo abierto en la ERS v2.0, sección 6.4.

3.6 Colección contactos

Almacena las consultas enviadas por los visitantes a través de la página de contacto.

Campo	Tipo	Obligatorio	Descripción y reglas
_id	ObjectId	Automático	Identificador único generado por la base de datos.
nombreCompleto	String	Sí	Nombre de quien envía la consulta.
correo	String	Sí	Correo de contacto para la respuesta.
asuntoContacto	String	Sí	Categoría de la consulta. Valores admitidos: Actividad, Inscripciones, Solicitud de cancelación de inscripción, Stands, Soporte Técnico.
mensaje	String	Sí	Contenido de la consulta.
fechaContacto	String	Sí	Fecha de envío de la consulta.

3.7 Relaciones entre colecciones



Relación	Cardinalidad	Implementación
actividad — inscripcion	1 : N	Cada inscripción referencia exactamente una actividad mediante actividadSeleccionada. Una actividad puede tener múltiples inscripciones.
stand — actividad	N : M	Cada stand mantiene un arreglo de referencias a actividades en actividadesStand. Una actividad puede estar vinculada a varios stands.
administrador	Independiente	Sin relaciones. Concentra los correos autorizados.
contacto	Independiente	Sin relaciones. Cada consulta es autocontenida.

4. Estructura de carpetas

El repositorio separa el backend y el frontend en dos directorios de primer nivel, y la documentación en un tercero. Dentro del backend, los modelos y las rutas residen en subdirectorios propios, en cumplimiento de RNF-MAN-01.

Campus-Fest-Grupo-3/

— README.md	Instalación, configuración y ejecución
— .gitignore	Excluye node_modules y .env
— BACK-END/	
— index.js	Punto de entrada: middleware, conexión y rutas
— package.json	Dependencias del proyecto
— .env	Variables de entorno (NO se versiona)
— models/	Esquemas de datos (Mongoose)
— actividad.model.js	
— administrador.model.js	
— contacto.model.js	
— inscripcion.model.js	
— stand.model.js	
— routes/	Endpoints y reglas de negocio
— actividad.route.js	
— administrador.route.js	
— agenda.route.js	
— contacto.route.js	
— inscripcion.route.js	
— stand.route.js	
— FRONT-END/	
— index.html	Sitio público
— actividades.html	
— detalle-actividad.html	
— agenda.html	
— stands.html	
— inscripcion.html	
— contacto.html	
— admin-login.html	Panel administrativo
— admin-dashboard.html	
— admin-actividades.html	
— admin-inscripciones.html	
— admin-stands.html	
— admin-contacto.html	
— admin-resultados.html	
— css/	
— styles.css	Hoja única: variables, componentes y modo oscuro
— js/	
— main.js	Utilidades comunes (menú, modales, notificaciones)
— proteger-ruta.js	Control de acceso al panel
— (un módulo por vista)	
— docs/	Documentación del proyecto
— ERS_CampusFest_v2.0	
— EDS_CampusFest_v2.0	
— Matriz_Trazabilidad_v3.0	
— Plan_de_Pruebas	

5. Diseño de la API REST

5.1 Convenciones

- Los recursos se nombran en minúscula y singular: /actividad, /inscripcion, /stand, /agenda, /contacto, /administrador.
- La operación se expresa mediante el método HTTP: POST crea, GET consulta, PUT modifica y DELETE elimina.
- El cuerpo de las peticiones y las respuestas se intercambia en formato JSON.
- Los filtros se transmiten como parámetros de consulta en la URL, no en el cuerpo.
- Los errores se devuelven con un objeto que contiene la propiedad mensajeError y un texto comprensible para el usuario final, según RNF-DIS-02.

5.2 Códigos de estado

Código	Significado	Uso en el sistema
200 OK	Operación completada	Consultas, modificaciones y eliminaciones exitosas.
201 Created	Recurso creado	Alta de actividad, inscripción, stand o consulta de contacto.
400 Bad Request	Petición inválida	Campos obligatorios ausentes, formato incorrecto, o regla de negocio incumplida (cupos llenos, inscripción duplicada, cierre de 36 horas, correo no institucional).
401 Unauthorized	No autorizado	Intento de inicio de sesión con un correo que no pertenece al dominio institucional.
404 Not Found	Recurso inexistente	Consulta, modificación o eliminación de un identificador que no existe.
500 Internal Server Error	Error del servidor	Fallo no previsto durante el procesamiento de la petición.

5.3 Catálogo de endpoints

Autenticación — /administrador

Método	Ruta	Requisito	Descripción
POST	/administrador/login	RF-AUT-01	Inicia sesión del administrador. Recibe el correo en el cuerpo de la petición y verifica que pertenezca al dominio @ucenfotec.ac.cr. Responde 200 con confirmación de acceso, 400 si el correo está ausente y 401 si el dominio no es institucional.

Actividades — /actividad

Método	Ruta	Requisito	Descripción
POST	/actividad	RF-ACT-01, RF-ACT-02	Crea una actividad. Valida la presencia de los campos obligatorios y exige el enlace cuando la actividad es virtual. Inicializa los cupos disponibles con el cupo máximo y el estado en Disponible.
GET	/actividad	RF-ACT-05, RF-ACT-07	Consulta el catálogo de actividades. Admite los filtros categoría, estado, esVirtual y fecha como parámetros de consulta. El filtro por fecha se resuelve sobre el rango

Método	Ruta	Requisito	Descripción
			completo del día.
GET	/actividad/:id	RF-ACT-08, RF-FE-03	Consulta una actividad concreta con toda su información, incluidos los requisitos y los cupos restantes. Responde 404 si el identificador no corresponde a ninguna actividad.
PUT	/actividad/:id	RF-ACT-03, RF-ACT-06	Modifica una actividad existente, incluida su cancelación mediante el cambio de estado. Los cambios se reflejan en el catálogo y en la agenda.
DELETE	/actividad/:id	RF-ADM-02	Elimina una actividad. Responde 404 si el identificador no existe.

Inscripciones — /inscripcion

Método	Ruta	Requisito	Descripción
POST	/inscripcion	RF-INS-02 a RF-INS-06, RF-INS-11, RNF-03	Registra una inscripción. Aplica, en orden, las siguientes validaciones: campos obligatorios, formato y dominio institucional del correo, existencia de la actividad, actividad no cancelada, cupos disponibles, margen superior a 36 horas respecto al inicio, y ausencia de inscripción previa del mismo estudiante en esa actividad. Al registrar, decrementa los cupos y actualiza el estado a Lleno si se agotan.
GET	/inscripcion	RF-INS-09, RF-ADM-03	Consulta las inscripciones registradas, ordenadas alfabéticamente por nombre, resolviendo el nombre de la actividad asociada.
DELETE	/inscripcion/:id	RF-INS-10	Cancela una inscripción de forma manual y libera el cupo en la actividad correspondiente, restableciendo su estado a Disponible si procede.

Stand — /stand

Método	Ruta	Requisito	Descripción
POST	/stand	RF-STA-01, RF-STA-05	Registra un stand con sus datos y, opcionalmente, las actividades vinculadas. Valida que la categoría pertenezca a la lista admitida.
GET	/stand	RF-STA-02, RF-STA-04	Consulta los stands registrados. Admite el filtro categoría como parámetro de consulta y resuelve los nombres de las actividades vinculadas.
GET	/stand/:id	RF-STA-01	Consulta un stand concreto con sus actividades vinculadas resueltas.
PUT	/stand/:id	RF-STA-03	Modifica un stand existente, incluidas sus actividades vinculadas.
DELETE	/stand/:id	RF-STA-03	Elimina un stand. El cambio se refleja en la página pública.

Agenda — /agenda

Método	Ruta	Requisito	Descripción
GET	/agenda	RF-AGE-01 a RF-AGE-05	Consulta la agenda del festival: las actividades ordenadas por fecha y hora, con su nombre, hora, lugar, categoría y estado. Admite los filtros fecha y categoría. El filtro por fecha se resuelve sobre el rango completo del día. Es de acceso

Método	Ruta	Requisito	Descripción
			público.

Contacto — /contacto

Método	Ruta	Requisito	Descripción
POST	/contacto	RF-CON-01, RF-CON-02, RF-CON-05	Registra una consulta enviada desde la página de contacto. Valida los campos obligatorios y que el asunto pertenezca a la lista admitida.
GET	/contacto	RF-CON-03	Consulta los mensajes de contacto recibidos, para su atención desde el panel administrativo.
DELETE	/contacto/:id	RF-CON-04	Elimina un mensaje de contacto ya atendido.

6. Validación, seguridad y manejo de errores

6.1 Estrategia de validación en dos niveles

El sistema valida los datos en el cliente y en el servidor. La validación en el cliente ofrece una respuesta inmediata y evita peticiones innecesarias; la validación en el servidor es la que garantiza la integridad, dado que un cliente puede ser manipulado. Ninguna regla de negocio reside exclusivamente en el cliente.

Nivel	Responsabilidad	Ejemplos
Cliente (js/)	Comprobar que los campos obligatorios estén completos y que los formatos sean plausibles antes de enviar la petición.	Campos vacíos en el formulario de inscripción; dominio del correo institucional; formato del correo en el formulario de contacto.
Ruta (routes/)	Aplicar las reglas de negocio y la coherencia entre entidades. Es el nivel decisorio.	Cupos disponibles; margen de 36 horas; inscripción duplicada; dominio institucional; existencia de la actividad referenciada.
Esquema (models/)	Garantizar la estructura y el tipo de los datos que llegan a la base de datos.	Campos obligatorios; listas de valores admitidos para categorías y estados; obligatoriedad condicional del enlace en actividades virtuales.

6.2 Manejo de errores

Cada operación de la API se ejecuta dentro de un bloque de captura de excepciones. Los errores previsibles se traducen a un código 400 o 404 con un mensaje comprensible; los imprevistos se registran en la consola del servidor y se responden con un código 500 y un mensaje genérico, sin exponer detalles internos. Este comportamiento responde a RNF-DIS-02.

En el cliente, los errores se presentan mediante el componente de notificación (toast), tanto para los fallos de validación como para la imposibilidad de contactar al servidor. Cuando una consulta no devuelve resultados, la vista muestra un mensaje explicativo en lugar de un área vacía.

6.3 Seguridad

Aspecto	Diseño implementado	Requisito
Protección del panel	El módulo proteger-ruta.js se carga en cada página administrativa y redirige al inicio de sesión si no existe una sesión activa en el almacenamiento del navegador.	RF-AUT-02, RNF-SEG-01
Autenticación	Inicio de sesión mediante correo institucional, sin contraseña. La sesión se conserva en el almacenamiento de sesión del navegador.	RF-AUT-01
Validación de entradas	Validación en los tres niveles descritos en 6.1, con normalización del correo (espacios y minúsculas) antes de compararlo.	RNF-SEG-03
Gestión de credenciales	La cadena de conexión a la base de datos y el puerto residen en el archivo .env, excluido del control de versiones mediante .gitignore.	RNF-SEG-04

Limitación de diseño reconocida: la sesión no se verifica en el servidor y los endpoints de escritura no exigen autenticación. El control de acceso opera únicamente en el cliente, por lo que resulta eludible. Se documenta como riesgo abierto en la ERS v2.0, sección 6.4, junto con la mitigación recomendada.

7. Diseño de la interfaz

7.1 Sistema de diseño

La interfaz se construye sobre una hoja de estilos única que define variables de color, tipografía y espaciado, junto con un conjunto de componentes reutilizables. Este enfoque garantiza coherencia visual entre las trece pantallas y permite implementar el modo oscuro redefiniendo únicamente las variables de color.

Elemento	Definición
Paleta principal	Azul institucional como color primario, con una escala de tonos para fondos, bordes y texto secundario.
Color de acento	Amarillo, empleado en las acciones principales y en los elementos destacados de la portada.
Colores semánticos	Verde para el estado Disponible, ámbar para Lleno y rojo para Cancelado, aplicados de forma consistente en tarjetas, agenda y detalle.
Tipografía	Una familia para títulos y otra para el cuerpo del texto, con una escala de tamaños definida por variables.
Modo oscuro	Implementado con la media query prefers-color-scheme, redefiniendo las variables de color sin duplicar reglas.

7.2 Componentes reutilizables

Componente	Función	Empleado en
Barra de navegación	Identidad del sitio, navegación principal y accesos a inscripción y panel. Colapsa en un menú desplegable en pantallas estrechas.	Todas las páginas públicas
Tarjeta de actividad	Presenta nombre, categoría, fecha, hora, lugar, cupo disponible y acceso al detalle.	Portada, catálogo
Tarjeta de stand	Presenta nombre, categoría, responsable, ubicación, descripción y actividades vinculadas.	Página de stands
Indicador de estado	Etiqueta con color semántico que comunica el estado de la actividad.	Agenda, catálogo, detalle
Formulario	Bloque con etiquetas asociadas, campos y mensajes de validación.	Inscripción, contacto, panel
Ventana modal	Contenedor superpuesto para las operaciones de alta y modificación del panel.	Panel administrativo
Notificación (toast)	Aviso temporal de confirmación o error.	Todas las operaciones de escritura
Barra lateral del panel	Navegación entre las secciones administrativas y cierre de sesión.	Panel administrativo
Acordeón	Bloque plegable para las preguntas frecuentes.	Página de contacto

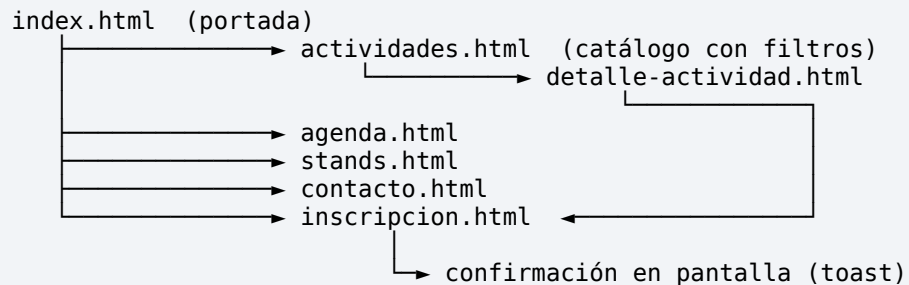
7.3 Diseño responsive

Punto de ruptura	Comportamiento	Requisito
1280 px o más	Navegación horizontal completa; tarjetas distribuidas en varias columnas; panel administrativo con barra lateral fija.	RF-RD-01

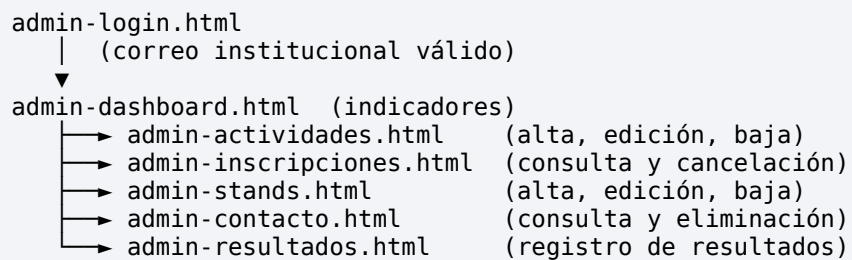
Punto de ruptura	Comportamiento	Requisito
Entre 900 y 1280 px	Reducción del número de columnas de las rejillas, conservando la navegación horizontal.	RF-RD-01
Hasta 900 px	Navegación colapsada en menú desplegable; rejillas a una columna; formularios a ancho completo; barra lateral del panel replegada. Sin desplazamiento horizontal a 375 px.	RF-RD-02

7.4 Flujo de navegación

SITIO PÚBLICO



PANEL ADMINISTRATIVO



Cualquier acceso directo sin sesión activa se redirige a `admin-login.html`

8. Trazabilidad del diseño

La siguiente tabla vincula cada módulo del diseño con los requisitos que satisface y los elementos que lo implementan. La correspondencia detallada por requisito individual se encuentra en la Matriz de Trazabilidad v3.0.

Módulo	Requisitos	Modelo	Ruta / API	Pantallas
Autenticación	RF-AUT-01, RF-AUT-02, RF-ADM-01, RNF-SEG-01	administrador.model.js	POST /administrador/login	admin-login.html, proteger-ruta.js
Actividades	RF-ACT-01 a RF-ACT-08, RF-ADM-02	actividad.model.js	/actividad (5 endpoints)	actividades.html, detalle-actividad.html, admin-actividades.html
Inscripciones	RF-INS-01 a RF-INS-11, RF-ADM-03, RNF-03	inscripcion.model.js	/inscripcion (3 endpoints)	inscripcion.html, admin-inscripciones.html
Stand	RF-STA-01 a RF-STA-05, RF-ADM-04	stand.model.js	/stand (5 endpoints)	stands.html, admin-stands.html
Agenda	RF-AGE-01 a RF-AGE-05, RF-FE-05	actividad.model.js (consulta)	GET /agenda	agenda.html
Contacto	RF-CON-01 a RF-CON-05, RF-FE-07	contacto.model.js	/contacto (3 endpoints)	contacto.html, admin-contacto.html
Resultados	RF-ADM-05	Pendiente	Pendiente	admin-resultados.html
Portada e indicadores	RF-FE-01, RF-ADM-06, RNF-02	—	GET /actividad, /stand, /inscripcion	index.html, admin-dashboard.html
Diseño responsive	RF-RD-01 a RF-RD-03, RNF-USA-02, RNF-USA-03	—	—	css/styles.css (todas las pantallas)

9. Decisiones de diseño registradas

Decisión	Alternativas consideradas	Justificación
Referenciar la actividad en las inscripciones en lugar de incrustar sus datos.	Incrustar el nombre y la fecha de la actividad en cada inscripción.	La actividad se consulta de forma independiente y sus datos cambian (estado, cupos). Referenciarla evita duplicación y actualizaciones en cascada.
Mantener el contador de cupos disponibles como campo de la actividad.	Calcular los cupos restantes contando las inscripciones en cada consulta.	El contador permite responder al catálogo y a la agenda sin agregaciones, y facilita el cambio automático de estado a Lleno.
Autenticación mediante correo institucional, sin contraseña.	Usuario y contraseña con almacenamiento encriptado.	El cliente indicó que no era obligatorio implementar autenticación avanzada. Reduce el alcance manteniendo la restricción de acceso al panel.
Estilos propios en lugar de un framework CSS.	Bootstrap, como se preveía en la ERS v1.0.	RF-RD-03 exige modo oscuro mediante media queries propias. Un sistema de variables propio lo resuelve sin sobrescribir estilos de terceros.
Un módulo JavaScript por vista.	Un único archivo compartido por todo el sitio.	Evita colisiones de identificadores entre vistas y acota el alcance de cada modificación, en línea con RNF-MAN-01.
Validación replicada en cliente y servidor.	Validar únicamente en el servidor.	La validación en el cliente mejora la respuesta percibida; la del servidor garantiza la integridad. Ninguna regla reside solo en el cliente.

10. Aprobación

Este documento constituye la versión final de la Especificación de Diseño de Software del proyecto CampusFest y describe el sistema tal como fue implementado al cierre del Sprint 4.

Rol	Nombre	Fecha
Equipo de desarrollo	Grupo 3 — Federico Barón Montoya, Steven Méndez Jiménez, Mariana Soto Castro	11 de agosto de 2026
Revisión académica	Verónica Isabel Mora Lezcano	